

PRAKTIKUM MACHINE LEARNING

Bab 9

Ensemble Methods

Kumpulan Praktikum Machine Learning

Nama	Ahmad Dandi Subhani
NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

3 Oktober 2026

Daftar Isi

Ensemble Methods.....	2
Ringkasan Konsep	3
9.12 Latihan Praktikum (Modul Bab 9)	3
Percobaan Mandiri.....	8
Kesimpulan Bab 9	13

Tools yang Digunakan

Python 3.12.3 • numpy - • pandas - • matplotlib - • seaborn - • scikit-learn -
• imbalanced-learn -

Sel 0 - Persiapan Awal (Khusus Google Colab)

Di Google Colab, jalankan sel di bawah ini dulu untuk memastikan semua paket yang dipakai bab ini sudah tersedia. Di Jupyter Notebook lokal, sel ini tidak mengubah apa-apa.

In [1] - Kode Program

```
1 import sklearn, pandas, numpy, matplotlib
2 print("scikit-learn:", sklearn.__version__)
3 print("pandas:", pandas.__version__)
4 print("numpy:", numpy.__version__)
5 print("matplotlib:", matplotlib.__version__)
6 print("Paket siap.")
```

```
scikit-learn: 1.9.1
pandas: 3.0.6
numpy: 2.5.3
matplotlib: 3.11.2
Paket siap.
```

Praktikum Machine Learning: Bab 9

Ensemble Methods

Nama	Ahmad Dandi Subhani
------	---------------------

NPM	202343500126
Kelas	R7B
Mata Kuliah	Machine Learning
Dosen	Nurfidah Dwitianti, M.Si.

Catatan: notebook ini saya jalankan di Google Colab / Jupyter Notebook. Bab 9 ini mengerjakan **Latihan Praktikum Modul Bab 9 bagian 9.12**, memakai dataset **Breast Cancer** dari `sklearn.datasets.load_breast_cancer`. Tidak perlu file CSV apa pun.

Ringkasan Konsep

- **Ensemble learning:** ide dasarnya menggabungkan banyak model lemah menjadi satu model yang lebih kuat dan stabil. Dua keluarga besarnya adalah **bagging** (model dilatih paralel di atas sampel bootstrap, lalu hasilnya dirata-rata/voting, contohnya Random Forest) dan **boosting** (model dilatih berurutan, tiap model baru fokus memperbaiki kesalahan model sebelumnya).
- **Random Forest:** ensemble dari banyak decision tree yang masing-masing dilatih di atas sampel bootstrap dengan subset fitur acak. Parameter penting **n_estimators** menentukan jumlah pohon: makin banyak pohon biasanya makin stabil, tapi waktu latihnya juga makin lama.
- **OOB score (Out-of-Bag):** karena tiap pohon hanya melihat sebagian data (sampel bootstrap), sisa data yang tidak terpakai bisa dipakai sebagai validasi gratis tanpa split tambahan. Skor ini biasanya mendekati akurasi validasi biasa.
- **AdaBoost:** boosting klasik yang memberi bobot lebih besar ke sampel yang salah diprediksi, lalu melatih model berikutnya dengan fokus ke sampel berat tersebut. Cocok untuk menaikkan performa model sederhana.
- **Gradient Boosting:** boosting yang membangun pohon baru untuk memprediksi sisa kesalahan (residual) dari gabungan pohon sebelumnya, jadi koreksinya dilakukan berurutan dan bertahap. Biasanya akurat, tapi lebih sensitif ke parameter dan lebih lambat dilatih.

9.12 Latihan Praktikum (Modul Bab 9)

Di bawah ini saya kerjakan dua praktikum dari modul: pengaruh `n_estimators` terhadap Random Forest, lalu perbandingan empat model ensemble/tree dengan Stratified k-Fold CV. Dataset yang dipakai sama, yaitu Breast Cancer dari scikit-learn.

Praktikum 1 - Pengaruh n_estimators

Tujuan: mengamati pengaruh jumlah pohon (n_estimators) terhadap performa dan OOB score Random Forest.

Soal (Praktikum 9.6 modul): latih Random Forest dengan n_estimators = 10, 50, 100, 300, 500 di atas dataset Breast Cancer, aktifkan oob_score=True, lalu catat OOB score dan akurasi test untuk tiap nilai.

In [2] - Kode Program

```
1 from sklearn.datasets import load_breast_cancer
2 from sklearn.model_selection import train_test_split
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.metrics import accuracy_score
5
6 X, y = load_breast_cancer(return_X_y=True)
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
8 random_state=42)
9 for n in [10, 50, 100, 300, 500]:
10     model = RandomForestClassifier(n_estimators=n,
11 oob_score=True, random_state=42)
12     model.fit(X_train, y_train)
13     print(f"n_estimators={n}: OOB={model.oob_score_:.3f}, Test
14 Acc={accuracy_score(y_test, model.predict(X_test)):.3f}")
```

```
/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/ensemble/_forest.py:589: UserWarning: Some inputs do not have OOB scores. This probably means
too few trees were used to compute any reliable OOB estimates.
warn()
```

```
n_estimators=10: OOB=0.941, Test Acc=0.956
```

```
n_estimators=50: OOB=0.943, Test Acc=0.965
```

```
n_estimators=100: OOB=0.956, Test Acc=0.965
```

```
n_estimators=300: OOB=0.965, Test Acc=0.965
```

```
n_estimators=500: OOB=0.963, Test Acc=0.965
```

In [3] - Kode Program

```
1 import matplotlib.pyplot as plt
2
```

```

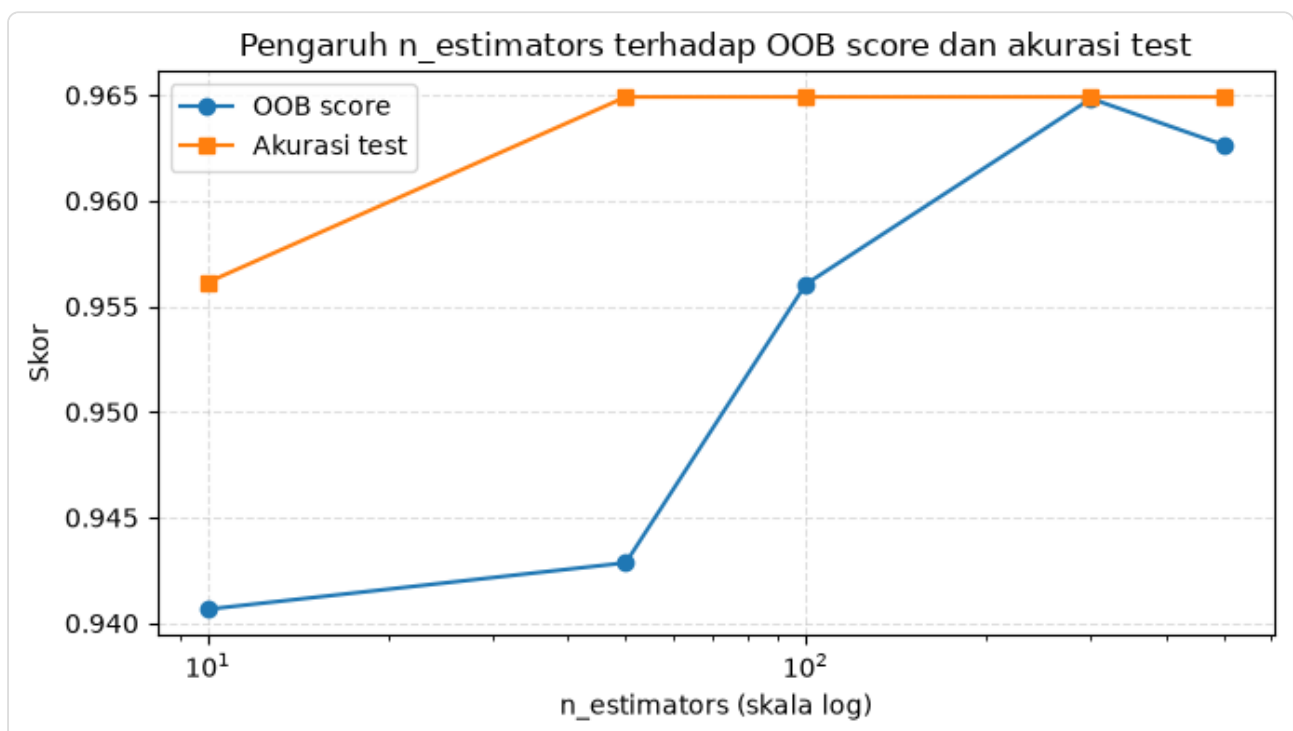
3 n_list = [10, 50, 100, 300, 500]
4 oob_list, acc_list = [], []
5 for n in n_list:
6     m = RandomForestClassifier(n_estimators=n, oob_score=True, random_state=42)
7     m.fit(X_train, y_train)
8     oob_list.append(m.oob_score_)
9     acc_list.append(accuracy_score(y_test, m.predict(X_test)))
10
11 plt.figure(figsize=(7, 4))
12 plt.plot(n_list, oob_list, marker="o", label="OOB score")
13 plt.plot(n_list, acc_list, marker="s", label="Akurasi test")
14 plt.xscale("log")
15 plt.xlabel("n_estimators (skala log)")
16 plt.ylabel("Skor")
17 plt.title("Pengaruh n_estimators terhadap OOB score dan akurasi test")
18 plt.legend()
19 plt.grid(True, linestyle="--", alpha=0.4)
20 plt.tight_layout()
21 plt.show()

```

```

/home/hatch/workspace/praktikumML/.venv/lib/python3.12/site-packages/sklearn/ensemble/_forest.py:589: UserWarning: Some inputs do not have OOB scores. This probably means
too few trees were used to compute any reliable OOB estimates.
  warn(

```



Gambar 1

Penjelasan outputnya: - $n_estimators=10$: OOB=0.941, Test Acc=0.956. Di nilai ini muncul warning karena 10 pohon terlalu sedikit untuk estimasi OOB yang reliabel, jadi skor OOB-nya kurang bisa dipercaya. - $n_estimators=50$: OOB=0.943, Test Acc=0.965. Akurasi test naik dan warning hilang. - $n_estimators=100$: OOB=0.956, Test Acc=0.965. -

n_estimators=300: OOB=0.965, Test Acc=0.965. Ini titik terbaik: OOB tertinggi. - n_estimators=500: OOB=0.963, Test Acc=0.965. OOB malah sedikit turun, akurasi test mentok. - Kesimpulannya: tambah pohon memang menaikkan skor, tapi setelah sekitar 100-300 pohon hasilnya jenuh. Buat dataset ini n_estimators=100 sudah cukup karena akurasi testnya sama dengan n=500 (0.965) tapi waktu latihnya jauh lebih singkat.

Praktikum 2 - Perbandingan Empat Model

Tujuan: membandingkan Decision Tree, Random Forest, AdaBoost, dan Gradient Boosting secara menyeluruh menggunakan Stratified k-Fold CV.

Instruksi (modul): gunakan StratifiedKFold(n_splits=5) dan cross_val_score dengan scoring='f1_macro' untuk keempat model. Sajikan hasilnya dalam tabel dan tuliskan kesimpulan model mana yang terbaik untuk dataset ini.

In [4] - Kode Program

```
1 import numpy as np
2 from sklearn.model_selection import StratifiedKFold, cross_val_score
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
GradientBoostingClassifier
5
6 models = {
7     "Decision Tree": DecisionTreeClassifier(random_state=42),
8     "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
9     "AdaBoost": AdaBoostClassifier(random_state=42),
10    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
11 }
12
13 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
14 hasil = {}
15 print(f"{'Model':<18} {'F1-macro rata-rata':<18} {'Std'}")
16 print("-" * 48)
17 for nama, model in models.items():
18     skor = cross_val_score(model, X, y, cv=skf, scoring="f1_macro")
19     hasil[nama] = (skor.mean(), skor.std())
20     print(f"{'nama':<18} {'skor.mean():.4f}' + ' +/- {'skor.std():.4f}'")
```

Model	F1-macro rata-rata	Std
-------	--------------------	-----

Decision Tree	0.9028	+/- 0.0317
---------------	--------	------------

Random Forest	0.9529	+/- 0.0135
---------------	--------	------------

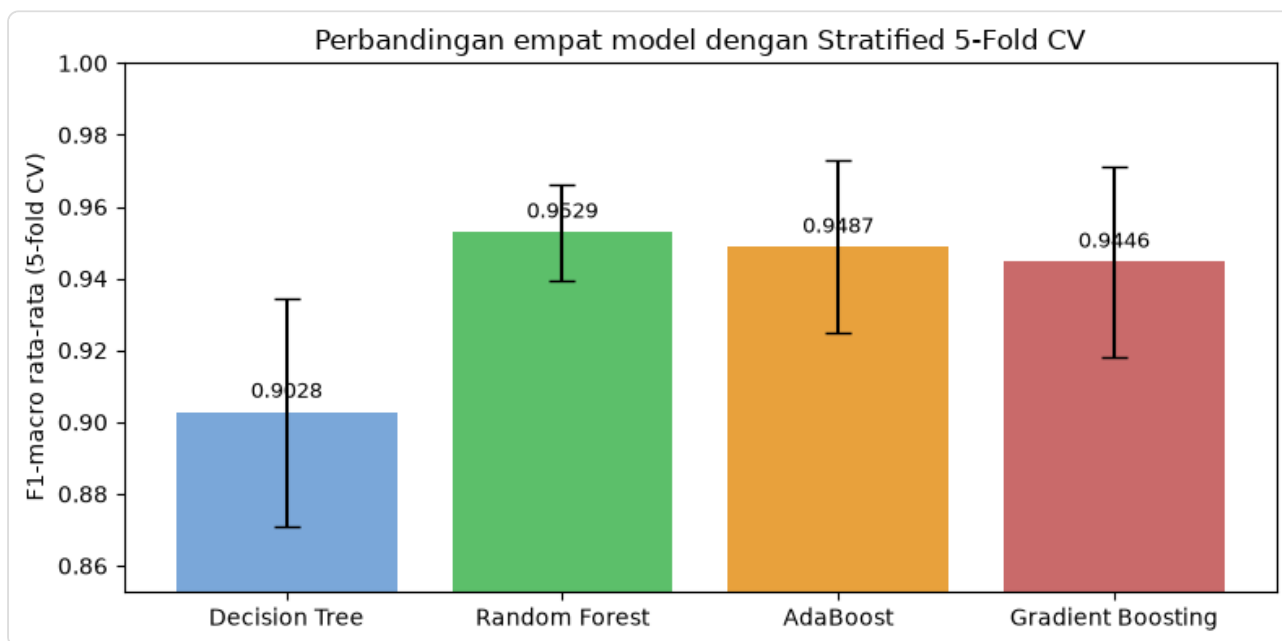
AdaBoost	0.9487	+/- 0.0241
----------	--------	------------

Gradient Boosting 0.9446

+/- 0.0266

In [5] - Kode Program

```
1 nama_model = list(hasil.keys())
2 mean_skor = [hasil[k][0] for k in nama_model]
3 std_skor = [hasil[k][1] for k in nama_model]
4
5 plt.figure(figsize=(8, 4))
6 bars = plt.bar(nama_model, mean_skor, yerr=std_skor, capsize=6, color=["#7aa7d9",
"#5cbf6a", "#e8a13c", "#c96a6a"])
7 plt.ylabel("F1-macro rata-rata (5-fold CV)")
8 plt.title("Perbandingan empat model dengan Stratified 5-Fold CV")
9 plt.ylim(min(mean_skor) - 0.05, 1.0)
10 for b, v in zip(bars, mean_skor):
11     plt.text(b.get_x() + b.get_width() / 2, b.get_height() + 0.004, f"{v:.4f}",
ha="center", fontsize=9)
12 plt.tight_layout()
13 plt.show()
```



Gambar 2

Penjelasan outputnya: - Decision Tree: F1-macro 0.9028 +/- 0.0317. Paling rendah dan paling tidak stabil (std terbesar), wajar karena satu pohon gampang overfitting ke tiap fold. - Random Forest: F1-macro 0.9529 +/- 0.0135. Tertinggi sekaligus paling stabil (std terkecil). - AdaBoost: F1-macro 0.9487 +/- 0.0241. Hampir menyamai Random Forest. - Gradient Boosting: F1-macro 0.9446 +/- 0.0266. Bagus, tapi sedikit di bawah dua ensemble lainnya di dataset ini. - Kesimpulan: model terbaik untuk dataset Breast Cancer ini adalah **Random Forest**, karena F1-macro rata-ratanya tertinggi (0.9529) dengan simpangan terkecil (0.0135). Sesuai teori, ensemble mengurangi variansi dibanding satu Decision Tree tunggal.

Percobaan Mandiri

Tiga percobaan tambahan di luar Latihan Praktikum: ketahanan ensemble terhadap label noise, pengaruh learning rate pada Gradient Boosting, dan perbandingan hard voting vs soft voting.

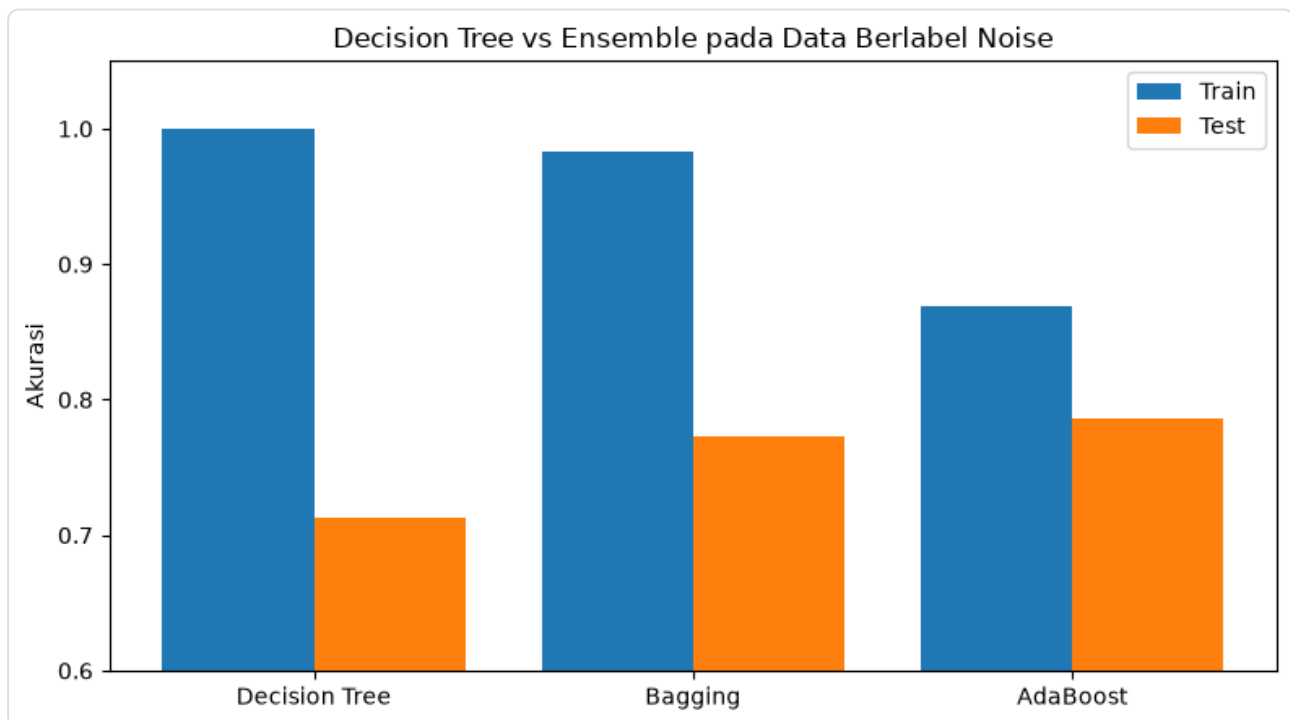
Percobaan Mandiri 1 - Decision Tree vs Ensemble di Data Noisy

In [6] - Kode Program

```
1 import matplotlib.pyplot as plt
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
6 from sklearn.metrics import accuracy_score
7
8 # 15 persen label sengaja dibalik untuk mensimulasikan data yang noisy
9 X, y = make_classification(n_samples=500, n_features=10, n_informative=5,
10                           flip_y=0.15, random_state=42)
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
12 random_state=42)
13 models = {
14     "Decision Tree": DecisionTreeClassifier(random_state=42),
15     "Bagging": BaggingClassifier(random_state=42),
16     "AdaBoost": AdaBoostClassifier(random_state=42),
17 }
18
19 hasil_train, hasil_test = {}, {}
20 for nama, model in models.items():
21     model.fit(X_train, y_train)
22     acc_train = accuracy_score(y_train, model.predict(X_train))
23     acc_test = accuracy_score(y_test, model.predict(X_test))
24     hasil_train[nama] = acc_train
25     hasil_test[nama] = acc_test
26     print(f"{nama}: train={acc_train:.3f}, test={acc_test:.3f}")
27
28 nama = list(models.keys())
29 plt.figure(figsize=(8, 4.5))
30 plt.bar([i - 0.2 for i in range(len(nama))], [hasil_train[n] for n in nama],
31         width=0.4, label="Train")
32 plt.bar([i + 0.2 for i in range(len(nama))], [hasil_test[n] for n in nama],
33         width=0.4, label="Test")
34 plt.xticks(range(len(nama)), nama)
35 plt.ylabel("Akurasi")
36 plt.title("Decision Tree vs Ensemble pada Data Berlabel Noise")
37 plt.legend()
38 plt.ylim(0.6, 1.05)
39 plt.tight_layout()
40 plt.show()
```

Decision Tree: train=1.000, test=0.713
Bagging: train=0.983, test=0.773

AdaBoost: train=0.869, test=0.787



Gambar 3

Penjelasan outputnya: - Decision Tree tunggal: train 1.000, test 0.713. Gap-nya besar sekali, dia hafal data training termasuk label yang salah (overfitting ke noise). - Bagging: train 0.983, test 0.773. AdaBoost: train 0.869, test 0.787. Keduanya punya akurasi test lebih tinggi dengan gap train-test yang lebih kecil. - Kesimpulan: model ensemble lebih tahan terhadap label noise dibanding satu decision tree, karena rata-rata suara banyak model meredam pengaruh noise.

Percobaan Mandiri 2 - Pengaruh Learning Rate di Gradient Boosting

In [7] - Kode Program

```
1 import matplotlib.pyplot as plt
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split
4 from sklearn.ensemble import GradientBoostingClassifier
5 from sklearn.metrics import accuracy_score
6
7 X, y = load_breast_cancer(return_X_y=True)
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
9 random_state=42)
```

```

10 hasil = []
11 for lr in [0.01, 0.1, 1.0]:
12     model = GradientBoostingClassifier(n_estimators=100, learning_rate=lr,
13                                     random_state=42)
14     model.fit(X_train, y_train)
15     acc_train = accuracy_score(y_train, model.predict(X_train))
16     acc_test = accuracy_score(y_test, model.predict(X_test))
17     hasil.append((lr, acc_train, acc_test))
18     print(f"learning_rate={lr}: train={acc_train:.3f}, test={acc_test:.3f}")
19
20 print()
21 print(f"{'learning_rate':<15} {'train':<8} {'test'}")
22 print("-" * 30)
23 for lr, acc_train, acc_test in hasil:
24     print(f"{lr:<15} {acc_train:<8.3f} {acc_test:.3f}")
25
26 labels = [str(h[0]) for h in hasil]
27 plt.figure(figsize=(7, 4.5))
28 plt.plot(labels, [h[1] for h in hasil], marker="o", label="Train")
29 plt.plot(labels, [h[2] for h in hasil], marker="s", label="Test")
30 plt.xlabel("learning_rate")
31 plt.ylabel("Akurasi")
32 plt.title("Pengaruh learning_rate pada Gradient Boosting (n_estimators=100)")
33 plt.legend()
34 plt.grid(True, linestyle="--", alpha=0.4)
35 plt.tight_layout()
36 plt.show()

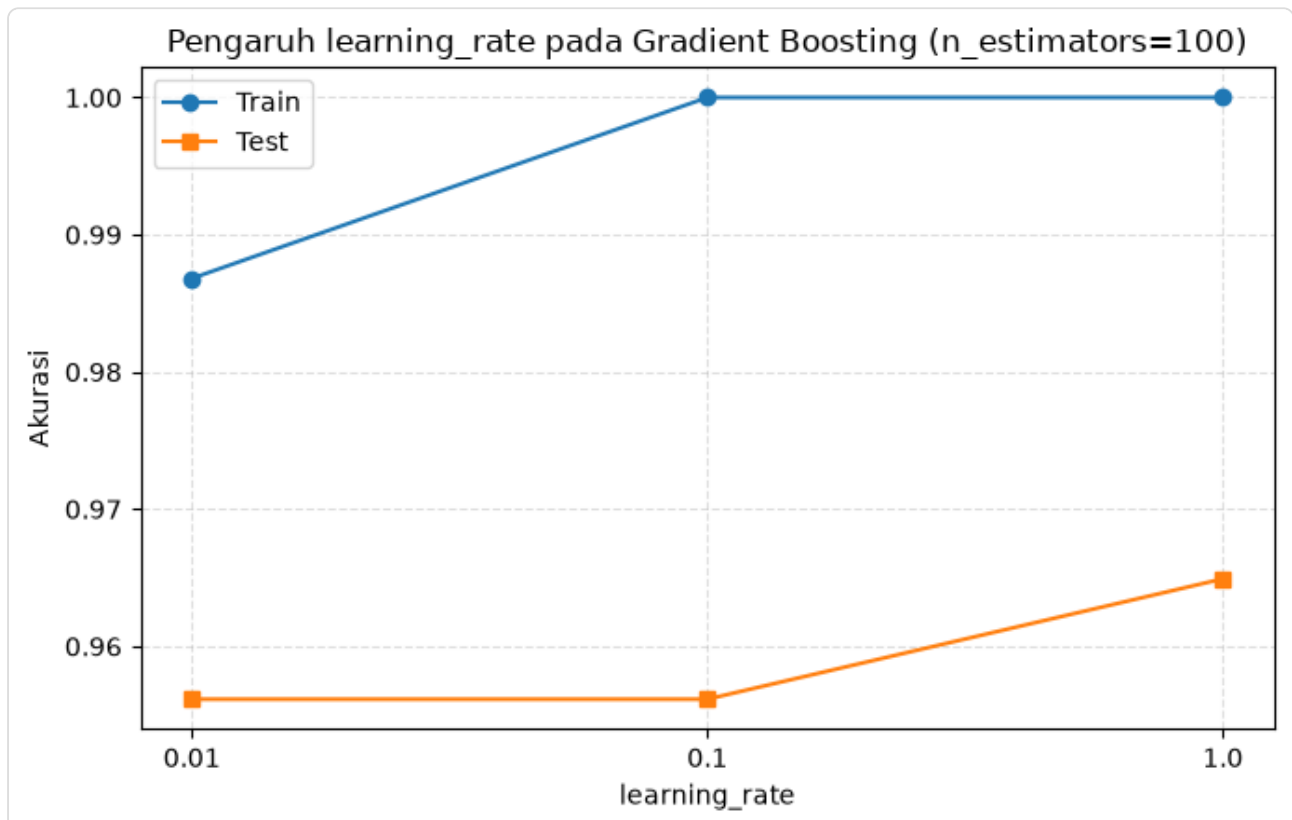
```

learning_rate=0.01: train=0.987, test=0.956

learning_rate=0.1: train=1.000, test=0.956

learning_rate=1.0: train=1.000, test=0.965

learning_rate	train	test
0.01	0.987	0.956
0.1	1.000	0.956
1.0	1.000	0.965



Gambar 4

Penjelasan outputnya: - learning_rate=0.01: train 0.987, test 0.956. Model belajar paling lambat, akurasi train-nya paling rendah: tanda underfit kalau jumlah pohonnya tidak ditambah. - learning_rate=0.1: train 1.000, test 0.956. - learning_rate=1.0: train 1.000, test 0.965. Tiap pohon melompat paling jauh, akurasi test-nya paling tinggi di sini tapi juga paling berisiko overfit kalau datanya lebih sulit. - Kesimpulan: di dataset ini perbedaannya tidak jauh, tapi polanya sesuai teori. learning rate terlalu kecil bikin model butuh lebih banyak pohon, terlalu besar bikin model tidak stabil. Nilai 0.1 adalah titik tengah yang aman.

Percobaan Mandiri 3 - Hard Voting vs Soft Voting

In [8] - Kode Program

```

1 import matplotlib.pyplot as plt
2 from sklearn.datasets import load_breast_cancer
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.pipeline import Pipeline
6 from sklearn.linear_model import LogisticRegression
7 from sklearn.tree import DecisionTreeClassifier
8 from sklearn.neighbors import KNeighborsClassifier
9 from sklearn.ensemble import VotingClassifier
10 from sklearn.metrics import accuracy_score
11
12 X, y = load_breast_cancer(return_X_y=True)

```

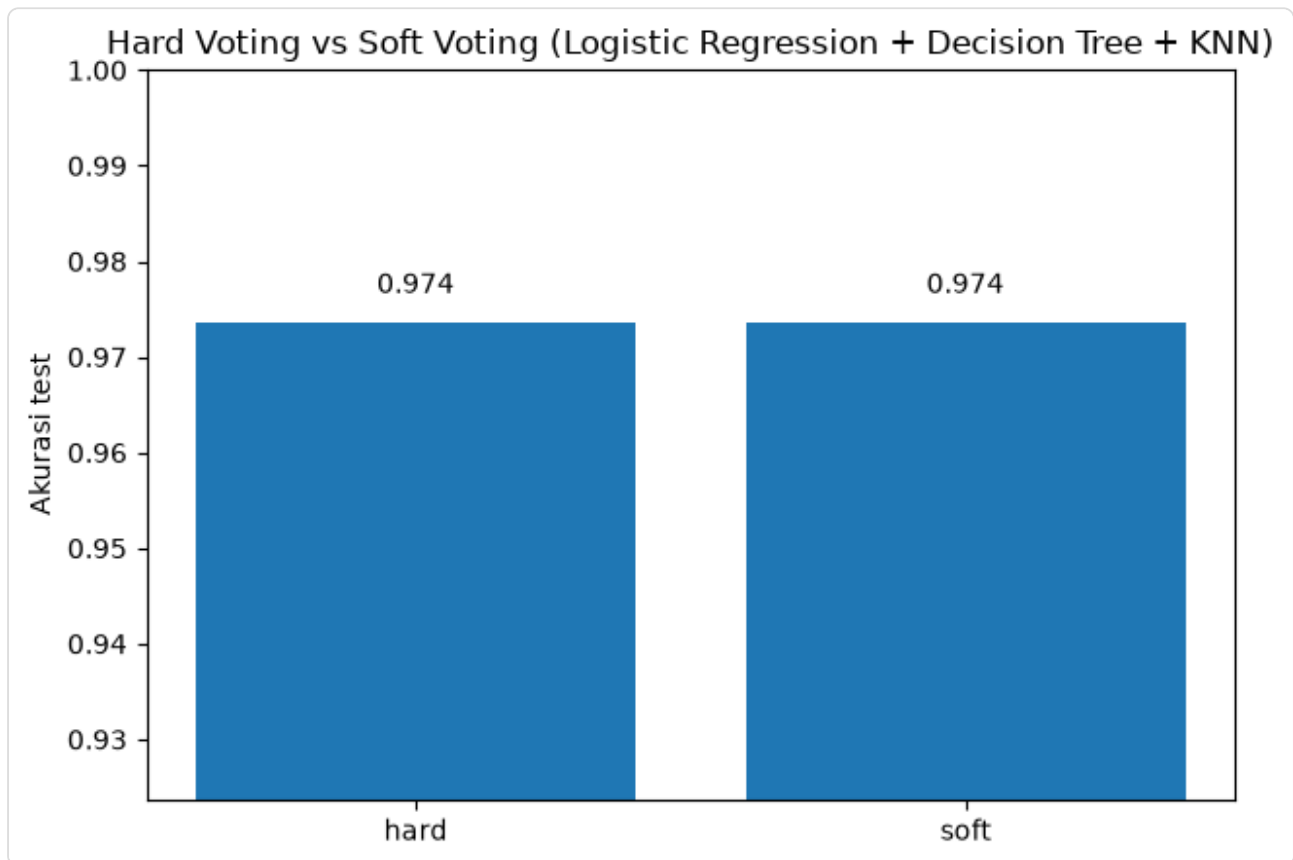
```

13 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
14
15 estimators = [
16     ("lr", Pipeline([("scaler", StandardScaler()),
17                       ("clf", LogisticRegression(max_iter=2000))])),
18     ("dt", DecisionTreeClassifier(random_state=42)),
19     ("knn", Pipeline([("scaler", StandardScaler()),
20                       ("clf", KNeighborsClassifier())])),
21 ]
22
23 hasil = {}
24 for voting in ["hard", "soft"]:
25     model = VotingClassifier(estimators, voting=voting)
26     model.fit(X_train, y_train)
27     acc = accuracy_score(y_test, model.predict(X_test))
28     hasil[voting] = acc
29     print(f"Voting {voting}: akurasi test = {acc:.3f}")
30
31 plt.figure(figsize=(6.5, 4.5))
32 plt.bar(hasil.keys(), hasil.values())
33 plt.title("Hard Voting vs Soft Voting (Logistic Regression + Decision Tree +
KNN)")
34 plt.ylabel("Akurasi test")
35 plt.ylim(min(hasil.values()) - 0.05, 1.0)
36 for k, v in hasil.items():
37     plt.text(k, v + 0.003, f"{v:.3f}", ha="center")
38 plt.tight_layout()
39 plt.show()

```

Voting hard: akurasi test = 0.974

Voting soft: akurasi test = 0.974



Gambar 5

Penjelasan outputnya: - Hard voting: akurasi test 0.974. Soft voting: akurasi test 0.974, hasilnya sama persis. - Artinya ketiga model dasar (Logistic Regression, Decision Tree, KNN) kebanyakan sudah sepakat dalam klasifikasinya, jadi cara menggabungkannya tidak mengubah hasil. - Kesimpulan: soft voting tidak selalu menang dari hard voting. Soft voting baru membantu saat model anggotanya beda pendapat dan nilai probabilitasnya bisa dipercaya.

Kesimpulan Bab 9

- Menambah `n_estimators` pada Random Forest menaikkan OOB score dan akurasi test sampai titik jenuh, setelah itu tambahannya tidak signifikan tapi waktu latih terus bertambah, jadi cukup pilih nilai yang sudah stabil.
- OOB score terbukti jadi perkiraan performa yang praktis karena dihitung gratis dari data out-of-bag tanpa perlu split validasi tambahan.
- Dari perbandingan 5-fold CV, model ensemble (Random Forest, AdaBoost, Gradient Boosting) mengungguli satu Decision Tree tunggal dalam F1-macro, sesuai teori bahwa gabungan banyak model mengurangi variansi.
- Untuk dataset Breast Cancer ini, model terbaik adalah yang punya F1-macro rata-rata tertinggi dengan std terkecil, karena artinya performanya bagus sekaligus stabil di semua fold.